addition, stakeholders are rarely capable of providing all the requirements up front, even if they do know them ahead of time.

Stakeholders in large organizations have often been trained to believe that they must provide a detailed list of all their requirements at the start of a project. In traditional Waterfall-style projects, once the requirements have been signed-off on, the project management process deliberately hinders any changes, corrections, or additions that users might want to make: this is known as *change control*. This dissuasive change-control process tends to result in very large sets of requirements full of everything the stakeholder can possibly think of, with many features being included just because they "might come in handy" someday.

It would be easy to take the feature requests that come out of such a process at face value. After all, shouldn't the users know better than anyone else what they want? Why not just note down what they want, and then do it? There are two very good reasons why this would be a bad idea.

First, the people who produce these requirements—the business sponsors, the end users, and so forth—are typically not well versed in the technologies that will be used to deliver the solution. The features they request may not be optimal from a technical viewpoint (there may be better ways of getting the job done), from a usability perspective (they are not UX experts), or from a financial one (you may be able to get the job done faster and cheaper using a different approach). The development team has a professional responsibility to propose the most appropriate solution for a given problem. But if the business goals are not clearly expressed and understood, you'll be hard put to suggest potentially more appropriate solutions.

But there's a second reason that's arguably even more important than the first. If you don't know why you need to deliver a feature in a particular way, you'll have no way of knowing whether this feature is still useful or relevant when change happens.

Any software project is an exercise in ongoing learning. Things inevitably change along the way, including assumptions and even the team's understanding of how a particular feature might benefit the business. A requirement that's expressed in the form of a detailed technical solution is embedded in a fabric of assumptions and preconceptions. You can be sure that some of these assumptions will turn out to be incorrect, or will change along the way, but it's hard to predict which ones will be wrong. When changes do happen, you need to be able to reassess the relevance of the features that you're building, but if you don't know why the business wants a feature, this will be much more difficult.

You usually need to drill down a little to discover what the business really needs. One of the best ways to do this is to repeatedly ask "why" until you get to a viable business goal. As a rule of thumb, five why-style questions are usually enough to identify the underlying business value (see figure 3.4).

Let's look at a practical example. Suppose you're working for a large media corporation. The director of marketing has come to you with a requirement that seems simple enough on the surface: "I want people to be able to post their printed classified ads online." To learn a little more, you might respond with something along the